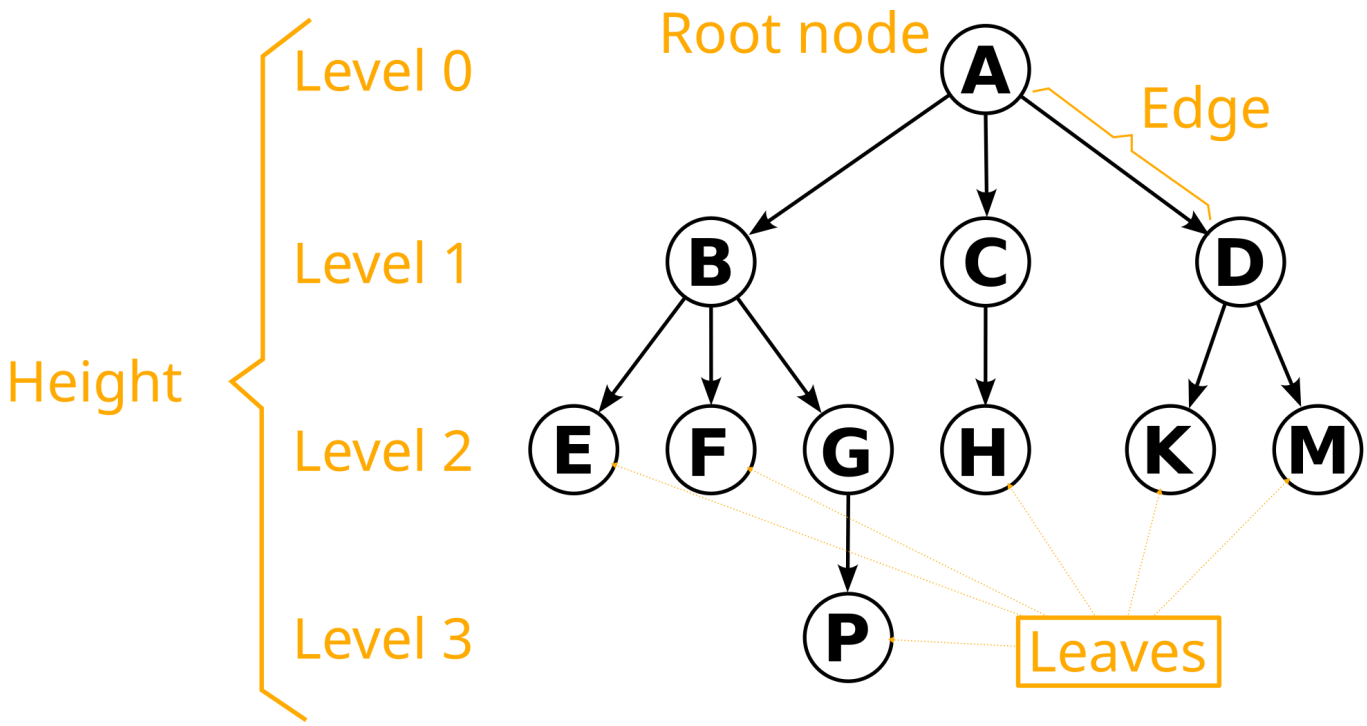
Trees

Until now we have seen some structures that somehow, we can consider 1-dimensional. You can move forward, or you can move backwards. **Trees** scale up things and now we can consider that trees are somehow 2-dimensional structures, where there can be multiple directions to follow. Trees have different components and rules too, to try to organize things a little bit. Let's see.

At the core of all these structures we are seeing, there is the **node**. As we have seen before, the node is a simple structure that contains data that has to be managed. A tree is a collection of nodes, where nodes are arranged like in a tree. There is a root, the **root node**, and from there comes one or more branches of nodes or **sub-trees**. So, nodes are connected to each other in the form of **parent-children** relationships. The root node is a parent that can have one or more children, and the same can be applied to every node in the tree. All the trees can have children. The number of children of a node is called the **degree** of the node. A node can have multiple children, but if it has no children, then it gets the name of **leaf**. Continuing with the family simile, two nodes that have the same parent are **siblings**. It could be even possible to use terms like **cousins** or **grandparent**, but they should be used with care.

Anyway, to be considered a Tree, a data structure must obey certain rules. First, the tree is really visualized as an **inverted tree**, where the Root node is at the top. A Tree has one and only one Root node. Nodes are connected from top to bottom. That is, nodes can't have children at a higher level.

Another rule is that branches or sub-trees are not connected. That also means that a node can't have two parents, only one.

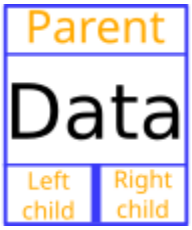


There are some important terms that Tree users should know. An **edge** is a connection between two nodes. The **level** or **depth** of a node is the number of edges between that node and the Root node. The different levels are easily visualized drawing the tree. The **Root node is at level 0**. The **height** of the tree is how many levels it has. In the image above, the height is 4 (levels from 0 to 3).

Binary Trees

Generally speaking, nodes of a Tree can have many (direct) children, but there is a type of Trees called **Binary Trees**, where there is a maximum of two children per node: **left child** and **right child**. This is the maximum; a node still can have one or no children. This kind of Tree are used very often, especially as they are the base for a certain kind of tree we'll see shortly. Besides of that rule, all the other rules of Trees still apply.

So, how would look a node for a Binary Tree? We are going to visually represent it like this:



And an implementation can be something like this:

```
class Node():
    def __init__(self, data, parent_node=None, left_child=None, right_child=None):
        self.data = data
        self._parent_node = parent_node
        self._left_child = left_child
        self._right_child = right_child
```

It's almost the same as the Node we have already seen with Doubly Linked Lists, but pointers names are different. Here a pointer to the parent node has been added, as it makes the Tree easier to traverse. But it's not mandatory and all we are going to see can also be done without it.

Binary Search Trees (BST)

One kind of special kind of Binary Tree is the **Binary Search Tree** or **BST**, that is like a **sorted** or **ordered** Binary Tree. This kind of Trees add one rule more: for each node, all nodes to its left have a smaller or equal value than itself, and all nodes to its right have a bigger value than itself. This rule allows to insert, delete and search through the tree very efficiently as we will see later. One thing to consider also is that in this kind of Trees, the values can be sorted and compared between them. This is easy if we are dealing with numeric values, but given different kind of values, the coder has to decide how they compare to each other. Maybe for text it would be alphabetically? Or is it better by the length of the text? It depends on the goals of the project. Also, as the Tree is ordered, there can't be duplicates, but if your case needs them, you can adapt your algorithms for them. For now we are going to use only numbers to make it easy.

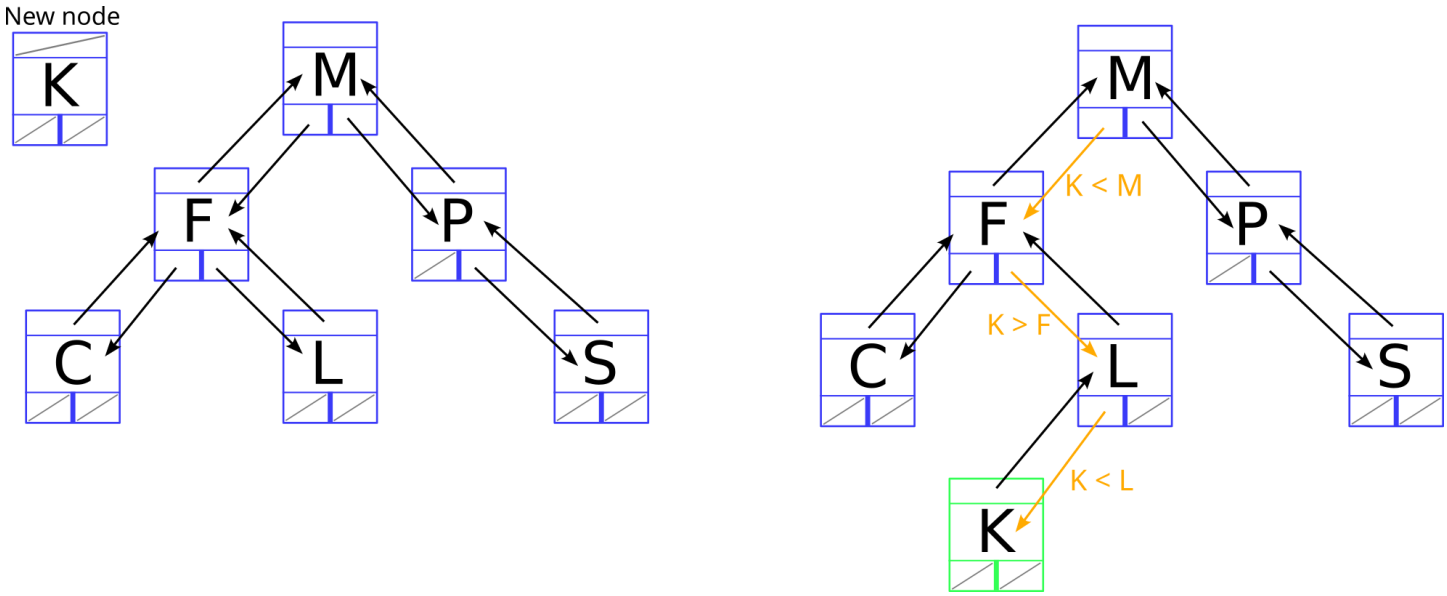
The benefit of a BST over a list is that for searching an element, in the list in the worst case you have to search through the entire list before finding the value. That makes it of time complexity $O(n)$. In a well-balanced tree, it depends on the height h of the tree, making it of time complexity $O(h)$. What is a well-balanced tree? Well, it's one where the nodes are distributed uniformly across as few levels as possible. For a Binary tree, the minimum number of levels is $\log_2 n$

We are going to implement a BST. The start is very simple, we just need a root node pointer:

```
class Tree():
    def __init__(self):
        self._root_node = None
```

Inserting a node into a BST

Now to populate the Tree, we need an insert method. The insert method should follow the rules of the BST, so to find where in the Tree to insert the new node, it will just, starting at the root node, compare the new value with the value of the current node, move to the left or right accordingly, and repeat the operation with every node in the path until there is no nodes to visit. The new values always are leaves with no subtrees.



Let's implement that!

```
    def __insert__(self, data):
        """
        Inserts a new value in the BST

        Parameters:
        - 'data': Value or data to insert

        Returns: None
        """
        # Let's use a couple of pointers to traverse the tree
        # following BST rules
        current_node = self._root_node
        parent_node = None
        while current_node:
            parent_node = current_node
            if data <= current_node.data:
                current_node = current_node._left_child
            else:
                current_node = current_node._right_child
```

(Continue...)

```
        # After the loop, parent_node points to the node that should be the
        # Parent node or None if Tree is empty
        new_node = Node(data, parent_node=parent_node)
        # Link parent node to new node.
        if parent_node is None:
            # If tree is empty, just make the new node the root node
            self._root_node = new_node
            return
        elif new_node.data < parent_node.data:
            # If value of new node is smaller than parent's, add new node to its left
            parent_node._left_child = new_node
        else:
            # If value of new node is bigger than parent's, add new node to its right
            parent_node._right_child = new_node
        return
```

That's kind of straightforward! 😃.

Notice that the method accepts a value and not a node. The node will be created internally. That way, the user using the class does not have to deal directly with the nodes.

Find a node based on a value

One action that the class Tree has to support is to find or search for a node based on its value. This should be a private method, as the class users should not directly manage nodes.

Search on a BST is very easy and fast due to the nature of the BST. Starting from the Root node, the method should first check that the current node is a valid node and then compare the value to be found and the value of the current node. If there is a match, return the current node. If not, compare where to continue to the left or to the right of the current node. Follow the pointer and repeat.

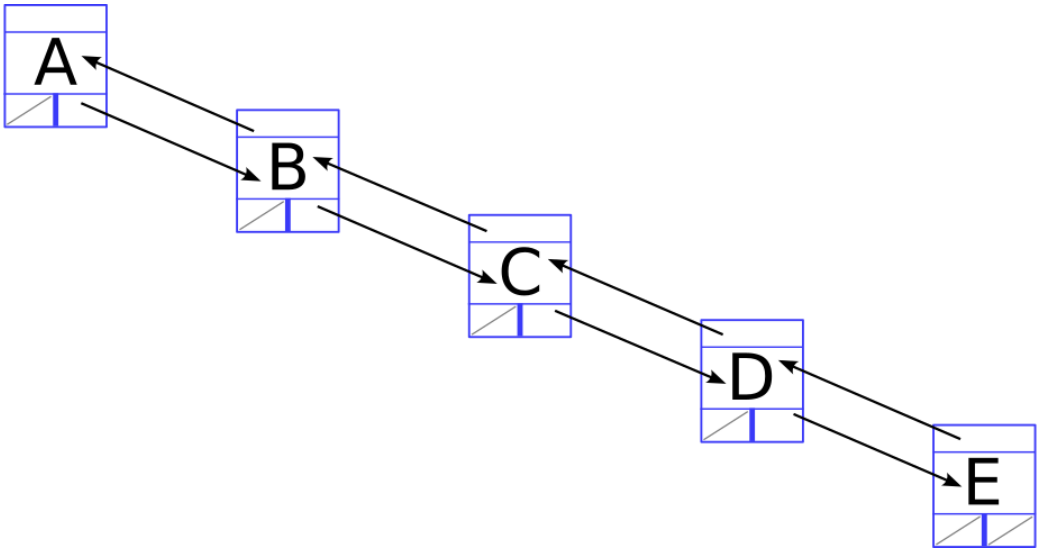
Exercise!

Open the Moodle environment and write a method that finds a node that holds a certain value.

Don't continue until you have finished the exercise!

Unbalanced trees

A possible problem with BSTs is that the Tree is "unbalanced". That means that instead of the nodes being distributed uniformly across as few levels as possible, the tree or a sub-tree has all or many of its nodes at only one of the sides. Depending on how unbalanced the Tree is, it can become quite inefficient. The problem is that depending on the data being inserted, the Tree can become unbalanced. For example, inserting already ordered values we get a highly unbalanced Tree instead of a more balanced and "helpful" Tree if the values are stored more or less randomly.



To solve this problem, there are some balance algorithms that are used in self-balancing trees. These structures will balance the tree at insertion and deletion. We will skip all the balancing stuff for the sake of simplicity but take in mind that you probably want to study that as well if you want to get into some more advanced subjects.

Find minimum and maximum values

Finding the minimum and maximum values in a BST is trivial. As BSTs are ordered, to find the minimum value, we just have to move down from Root node to the left until there are no more nodes. That last node is the minimum. To find the maximum we do the opposite. We mode down from Root node to right until there are no more nodes. That last node is the maximum.

Exercise!

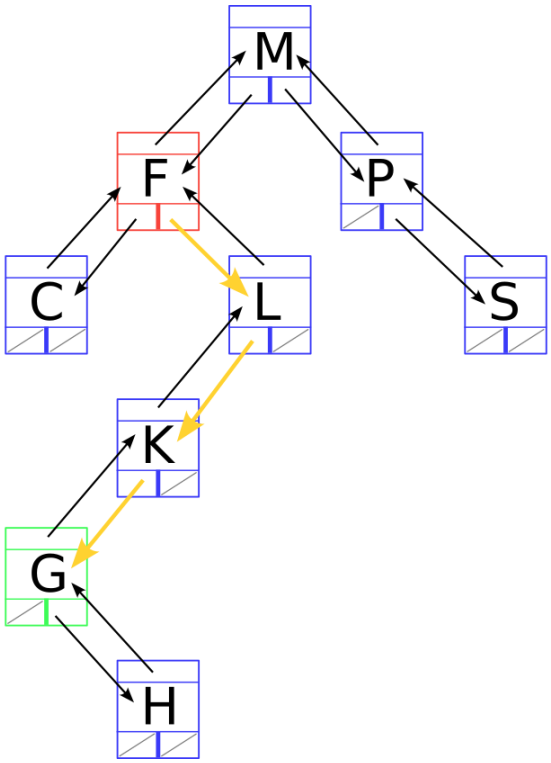
Open the Moodle environment and write two methods: one to find the maximum value of the tree and another one to find the minimum value of the tree.

Don't continue until you have finished the exercise!

Successor

Before we dig into deletion, there are a couple of concepts we need to know.

The first one is what a **Successor** is. The successor of a node is the node with next bigger value of all the nodes of the Tree. It can be found moving from the node we want to find the successor to its right child and then following the left child pointers until the end.



Implementation is quite easy:

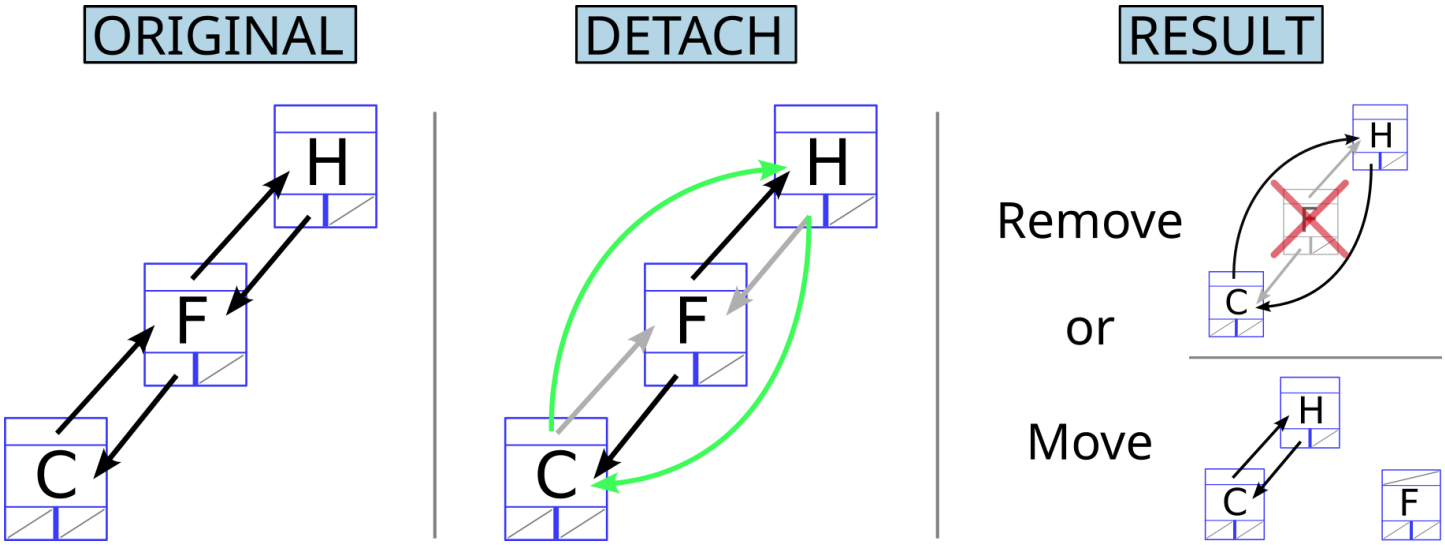
```
    def _find_successor(self, node):
        """
        Find the successor of a node

        Parameters:
        - 'node': The node for whom a successor has to be found.

        Returns: Returns the succesor node or None if no successor is found.
        """
        current = node._right_child
        while current._left_child:
            current = current._left_child
        return current
```

Detach a node

Another concept we need to know is what is to **detach** a node from the tree. This is not an official concept, but I think it describes quite well what we want to achieve sometimes.

To detach a node is to rearrange pointers so, the node is actually out of the tree. No pointers in the tree points to this node. The pointers that were pointing to this node (from the child or from the parent) are now pointing to its parent or child.

To be able to detach a node from the tree, it has to have **at most one child**, because it's not clear how to organize nodes if it had two children. After detaching the node, it can be removed from memory or moved to another position.



Notice

The image shows a node F being detached from the tree. It has a left child, but the same process should be done if the child were its right child or there were no children at all. Also, this process should succeed even if the node F have no parent (node F is the Root node)

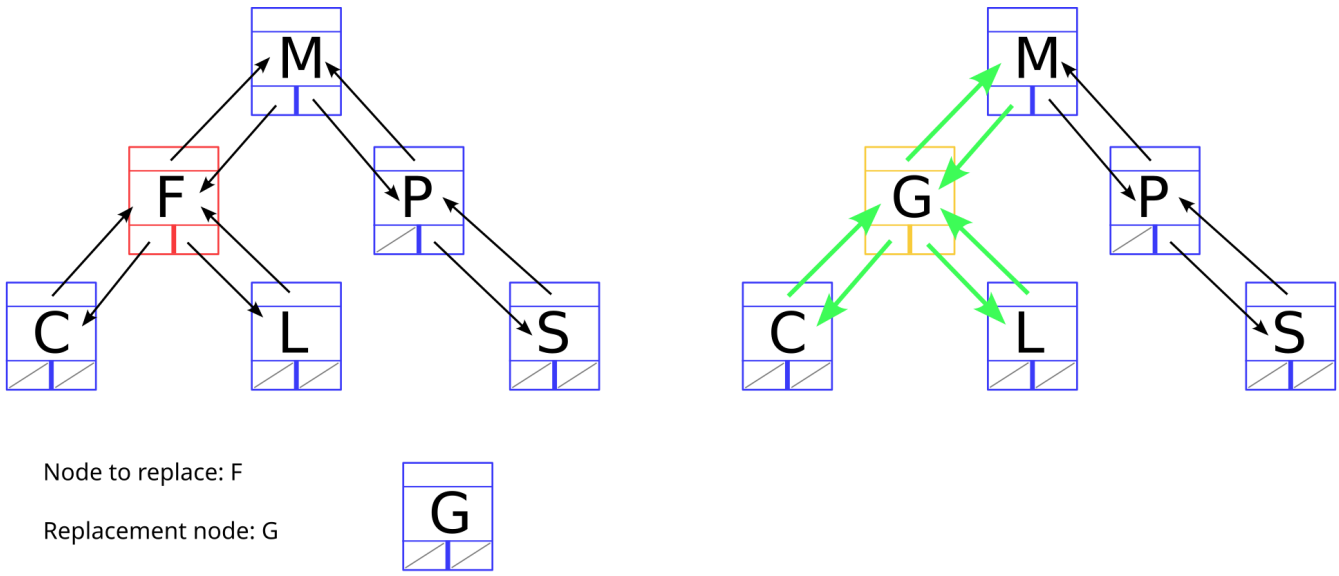
Exercise!

Open the Moodle environment and write a method that detach a node from the tree.

Don't continue until you have finished the exercise!

Replace a node with another one

An action that we are also going to need to be able to delete nodes, is to replace a node with another one.



As seen in the image, the idea is to update the pointers in the nodes involved, so the new node is put in place of the old node. To make the method more helpful, it has to manage some special cases:

- Parent node is None. That means that the node to be replaced is the Root node itself.
- Node to be replaced has two children or one child or no children at all.

Other than that, this procedure only requires to carefully change pointers in the affected nodes.

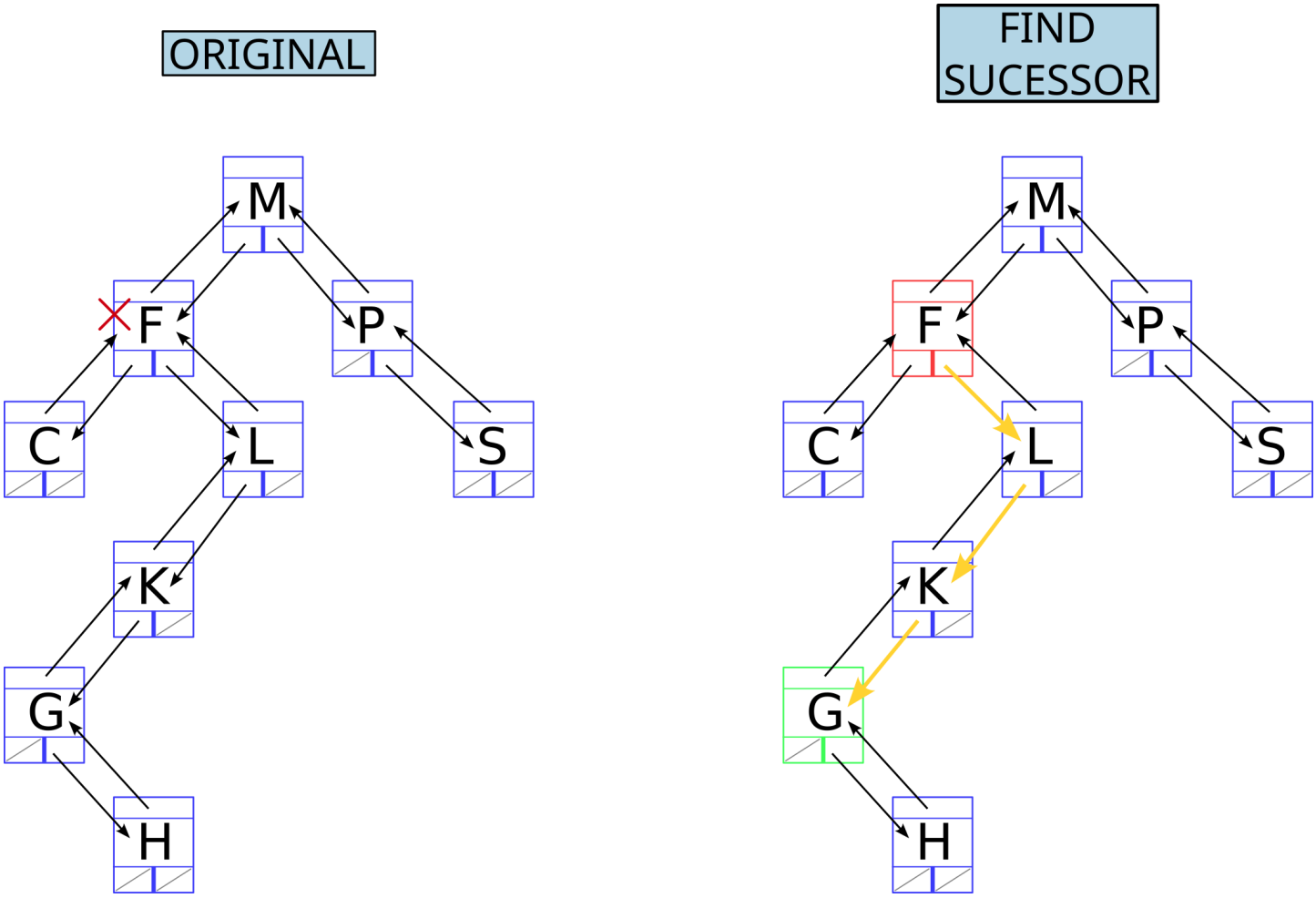
```python
    def _replace_node(self, node_to_replace, replacement_node):
        """
        Link the parent and children of the node to be replaced to the replacement node.
        Replacement node and node to be replaced must exist.
        Node to be replaced is not modified.
        If node_to_replace is Root node, then _root_node pointer is updated.
        if BST rules are not fulfilled, an error is thrown.
        """
        # Check nodes exist.
        if node_to_replace is None or replacement_node is None:
            raise(ValueError)

        parent_node = node_to_replace._parent
        # Link the replacement node to the parent
        replacement_node._parent = parent_node          # Bottom up

        # If node to replace is Root node, update _root_node pointer.
        if node_to_replace is self._root_node:
            self._root_node = replacement_node              # From top to bottom
        # If not, link parent to the replacement on the right or the left
        elif parent_node._left_child is node_to_replace:
            # Replacement is left node
            if replacement_node.data > parent_node.data:
                raise(ValueError)
            parent_node._left_child = replacement_node     # From top to bottom
        else:
            # Replacement is right node
            if replacement_node.data < parent_node.data:
                raise(ValueError)
            parent_node._right_child = replacement_node      # From top to bottom

        # Link replacement node to child nodes (if any)
        # From parent to child
        replacement_node._left_child = node_to_replace._left_child
        replacement_node._right_child = node_to_replace._right_child
        # From child to parent
        if replacement_node._left_child:
            if replacement_node._left_child.data > replacement_node.data:
                raise(ValueError)
            replacement_node._left_child._parent = replacement_node
        if replacement_node._right_child:
            if replacement_node._right_child.data < replacement_node.data:
                raise(ValueError)
            replacement_node._right_child._parent = replacement_node
```

Deleting in a BST

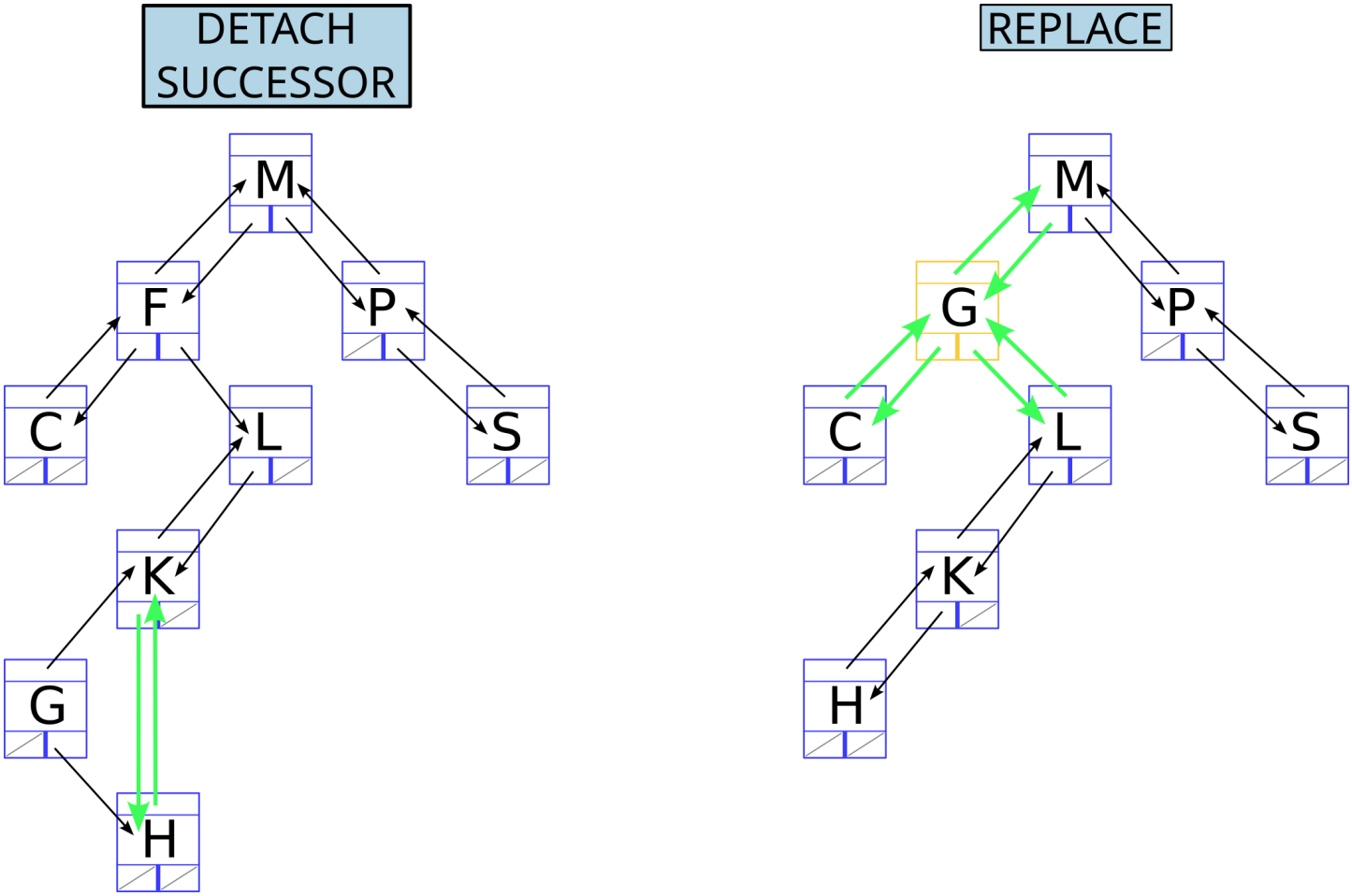
Deletion of a node is a bit of a complex case. But thanks to some of the helper methods that we have wrote beforehand, it is going to be quite easy to implement. There are two scenarios that have to be taken care of:

1. Node to remove has only one or no children. Just detach the node from the tree as seen before.
2. And the more complex case is when the node to remove has two children. The steps in this case would be:
    - Find the successor that "replaces" the node to be removed
    - Detach the successor from the tree to be able to move the successor to its new position. Successor's parent and child have to be linked between them.
    - And replace the node to be removed. The successor takes its place and it's linked with the parent and children.

This process, although it may seem complicated, is not so much. It's just a matter of being careful on each step of the process. We can see it visually like this:



(Continue in next page...)



The code can be seen next. The method uses quite literally the explained process.

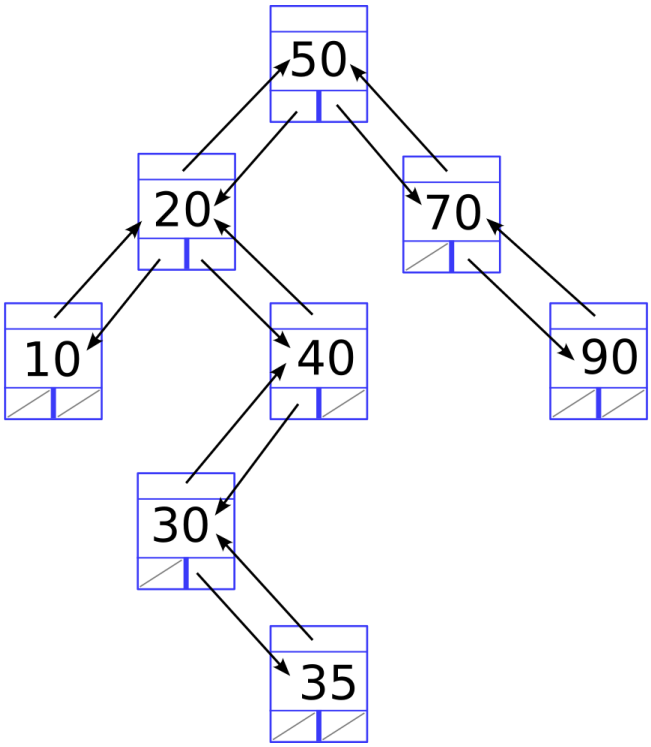
```
    def remove(self, data):
        # Find the node to remove
        node_to_remove = self._find(data)
        # if node is not found, return
        if not node_to_remove:
            return
        # If node has only one or no child, just detach the node from the tree,
        # replacing it with one of its childs (if any)
        if node_to_remove._left_child is None or node_to_remove._right_child is None:
            self._detach_node(node_to_remove)
        else:
            # Node to be removed has two children. Find its successor.
            # By definition the successor does not have a left child
            # (because then it would be the actual successor)
            successor_node = self._find_successor(node_to_remove)
            # Detach the successor from the tree
            self._detach_node(successor_node)
            # And replace the node to remove with the successor
            self._replace_node(node_to_remove, successor_node)
```

It has to be noted that usually this process is explained with more stress on the links and pointers changes and without helper methods. But I have opted here to show you a more general idea of what is happening, instead of focusing on the small details.

If you want to give it a try that way, you can complete the optional exercise on the Moodle environment. You can use the algorithm (with pseudocode) in the Wikipedia page about Binary Search trees.

Tree traversal

One interesting matter about trees is how to traverse a tree. We can traverse a tree in different ways. Let's see how the different forms work with an example tree:



- **Breadth-first traversal**: Tree is traversed by levels and from left to right. Starting on level 0 (Root node) and going down one level at a time traversing all nodes of a level before continuing to the next.
  So, for the example image, it would be: 50, 20, 70, 10, 40, 90, 30, 35
- **Depth-first traversal**: Tree is traversed in sub-trees. That means that starting from root different sub-trees are formed, if possible, and those sub-trees are traversed before continuing with the upper-level sub-trees. There are 3 possible ways of do it:
  - **In-order** traversal: For every sub-tree, the nodes are mark as visited from left to right: left child, parent and right child.
    For the example image, result would be: 10, 20, 30, 35, 40, 50, 70, 90
  - **Pre-order** traversal: For every sub-tree, the nodes are mark as visited in this order: parent, left child and right child.
    For the example image, result would be: 50, 20, 10, 40, 30, 35, 70, 90
  - **Post-order** traversal: For every sub-tree, the nodes are mark as visited in this order: right child, left child and parent.
    For the example image, result would be: 10, 35, 30, 40, 20, 90, 70, 50

Let's see their implementations.

The **Breadth-First** method uses a queue to traverse the tree horizontally. First enqueue the Root node and then, in a loop, get the next value of the queue, store the node's content (to a list, for example) and enqueue the left child and right children (if they exist). When there are no more nodes in the queue, the tree has been traversed completely.

```
    def breadth_first_traversal(self):
        values = []
        # Use a queue to store nodes to traverse
        queue = Queue()

        # Start by enqueuing the Root node
        queue.enqueue(self._root_node)

        # Continue while there is a node in the queue
        while node := queue.dequeue():
            # Save the value of the node
            values.append(node.data)

            # Enqueue the left child
            if node._left_child:
                queue.enqueue(node._left_child)

            # Enqueue the right child
            if node._right_child:
                queue.enqueue(node._right_child)

        # Return values
        return values
```

The **Depth-First** methods are quite similar to each other. For the implementation of these methods, we are going to use recursion. Just in case... recursion is when a function calls itself to solve a problem. This is a very typical way of solving things in some cases. The function must have a base case that is triggered at some point and that avoids that the function continues to call itself indefinitely. Recursion can be a little bit more challenging to master, but it's kind of the natural way of solving some problems and recursive functions are usually shorter than their iterative counterparts.

These methods basically just save the node value and call themself for each child in the order that corresponds to the method.

```
    def depth_first_in_order(self, root_node=None, values=[]):
        """
        Traverse the tree in Depth-first in-order

        Parameters:
        - 'start_node': Optional. Where to start the traversal
        - 'values': Optional. A list of already visited nodes. For internal use only.

        Returns: A list of visited nodes in Depth-first in-order
        """
        # If no start node, start from the Root node
        if start_node is None:
            current = self._root_node
        else:
            current = start_node

        # Visit left child, if exists
        if current._left_child:
            self.depth_first_in_order(current._left_child, values)

        # Add current node value
        values.append(current.data)

        # Visit right child, if exists
        if current._right_child:
            self.depth_first_in_order(current._right_child, values)

        # Return list of visited nodes
        return values
```

```python
    def depth_first_pre_order(self, root_node=None, values=[]):
        """
        Traverse the tree in Depth-first pre-order

        Parameters:
        - 'start_node': Optional. Where to start the traversal
        - 'values': Optional. A list of already visited nodes. For internal use only.

        Returns: A list of visited nodes in Depth-first pre-order
        """
        # If no start node, start from the Root node
        if start_node is None:
            current = self._root_node
        else:
            current = start_node

        # Add current node value
        values.append(current.data)

        # Visit left child, if exists
        if current._left_child:
            self.depth_first_pre_order(current._left_child, values)

        # Visit right child, if exists
        if current._right_child:
            self.depth_first_pre_order(current._right_child, values)

        # Return list of visited nodes
        return values
```

```python
    def depth_first_post_order(self, root_node=None, values=[]):
        """
        Traverse the tree in Depth-first post-order

        Parameters:
        - 'start_node': Optional. Where to start the traversal
        - 'values': Optional. A list of already visited nodes. For internal use only.

        Returns: A list of visited nodes in Depth-first post-order
        """
        # If no start node, start from the Root node
        if start_node is None:
            current = self._root_node
        else:
            current = start_node

        # Visit left child, if exists
        if current._left_child:
            self.depth_first_post_order(current._left_child, values)

        # Visit right child, if exists
        if current._right_child:
            self.depth_first_post_order(current._right_child, values)

        # Add current node value
        values.append(current.data)

        # Return list of visited nodes
        return values
```

They could be a little simpler, but they are done this way, so the class user does not have to deal with nodes.